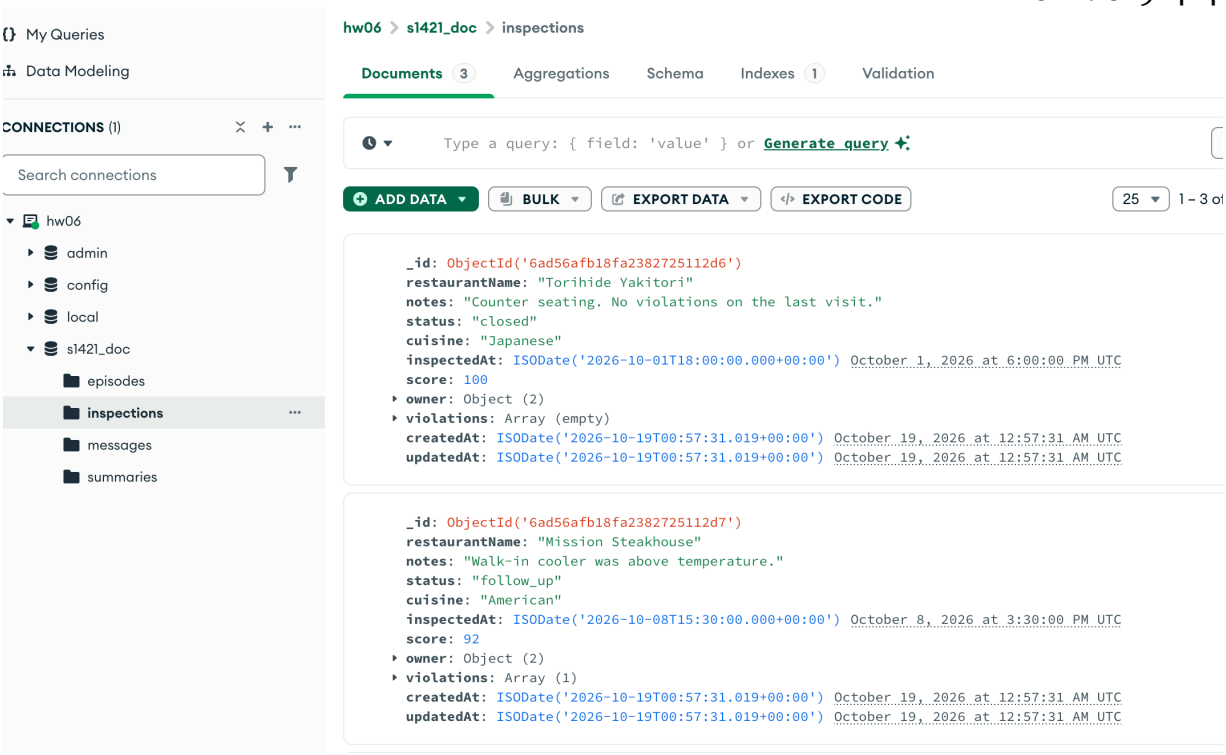
Data 260 – HW6
Eric Zhao
SID: 019141421

SID4: 1421
PORT_BASE: 8521
PREFIX: s1421
SEED: 1421
VERIFY_SEED: 2614221
DOMAIN_ID: 5
Hardware: Apple M2 Pro, 16GB
Local model used:
Git link: https://github.com/ericezhao/data260-1421
Tag link: https://github.com/ericezhao/data260-1421/tree/hw6
Tagged commit hash: 4128147fa80bfbef9c001b3891a47ed0b84fec78

Part 1



Data 260 – HW6
Eric Zhao
SID: 019141421




```
INSPECTION_VALIDATOR = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["restaurantName", "status", "cuisine", "inspectedAt",
"createdAt", "updatedAt"],
        "properties": {
            "restaurantName": {"bsonType": "string", "minLength": 1, "maxLength":
100},
            "notes": {"bsonType": "string", "maxLength": 2000},
            "status": {"enum": STATUSES},
            "cuisine": {"enum": CUISINES},
            "inspectedAt": {"bsonType": "date"},
            "score": {"bsonType": ["int", "long"], "minimum": 0, "maximum": 100},
            "owner": {
                "bsonType": "object",
                "required": ["name"],
                "properties": {
                    "name": {"bsonType": "string", "minLength": 1, "maxLength": 100},
                    "phone": {"bsonType": "string", "maxLength": 20},
                },
            },
            "violations": {
                "bsonType": "array",
                "items": {
                    "bsonType": "object",
```

```
                    "required": ["description", "severity"],
                    "properties": {
                        "description": {"bsonType": "string", "minLength": 1,
"maxLength": 300},
                        "severity": {"enum": SEVERITIES},
                    },
                },
            },
            "createdAt": {"bsonType": "date"},
            "updatedAt": {"bsonType": "date"},
        },
    }
}
```

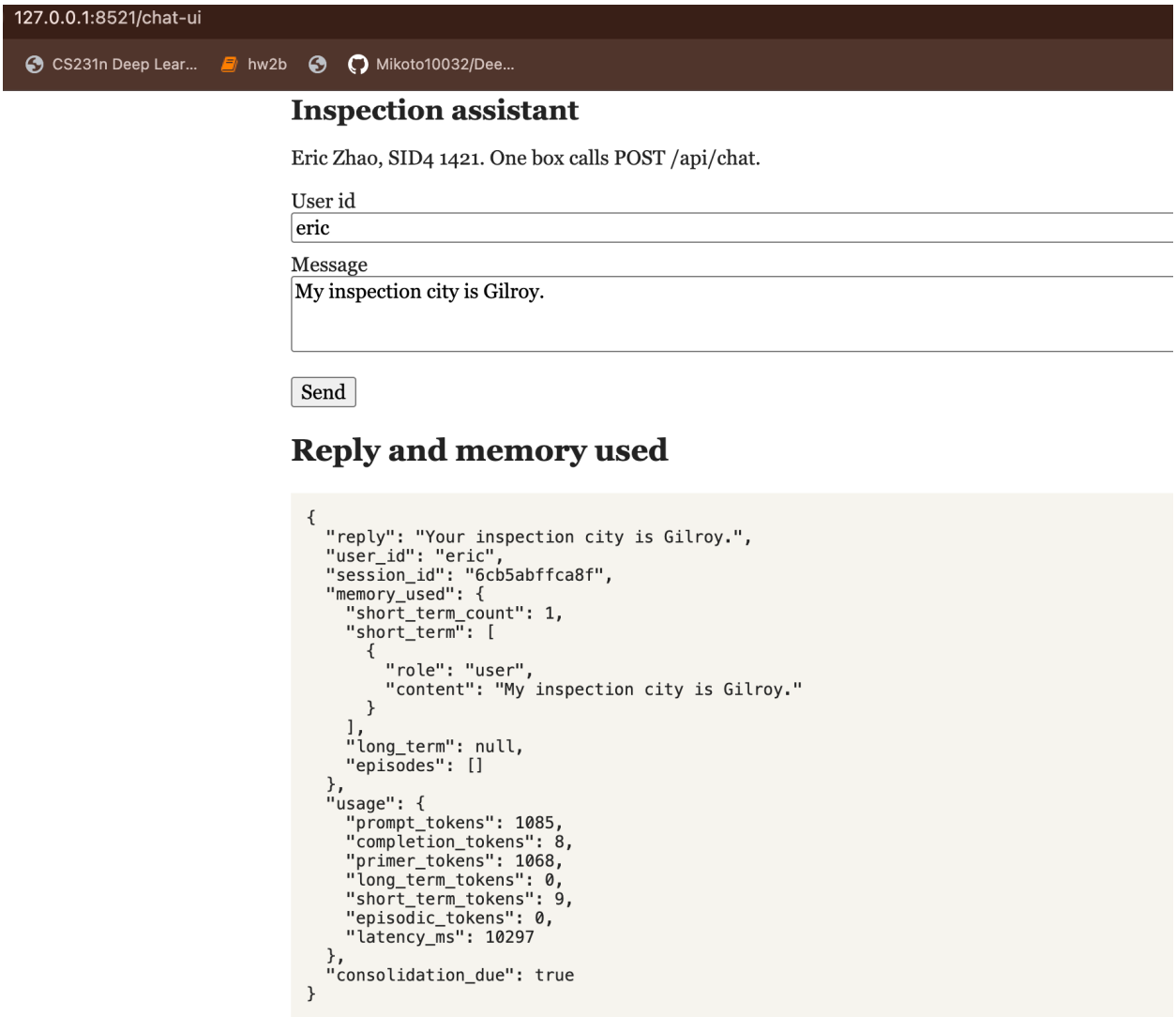
```
class InspectionDocCreate(BaseModel):
    restaurantName: str = Field(..., min_length=1, max_length=100)
    notes: Optional[str] = Field(None, max_length=2000)
    status: StatusEnum = "open"
    cuisine: CuisineEnum
    inspectedAt: datetime
    score: int = Field(100, ge=0, le=100)
    owner: Optional[OwnerDoc] = None
    violations: list[ViolationDoc] = []
```

```
@router.post("", response_model=InspectionDocOut, status_code=status.HTTP_201_CREATED)
def create_document(payload: InspectionDocCreate):
    _ready()
    col = inspections()
    doc = payload.model_dump(exclude_none=True)
    ts = now_utc()
    doc["createdAt"] = ts
    doc["updatedAt"] = ts
    try:
        res = col.insert_one(doc)
        created = col.find_one({"_id": res.inserted_id})
        return serialize_inspection(created)
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while creating the
document.")


@router.get("", response_model=list[InspectionDocOut])
def list_documents():
    _ready()
    col = inspections()
    try:
```

```
        docs = list(col.find().sort("createdAt", -1))
        return [serialize_inspection(doc) for doc in docs]
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while listing
documents.")
```

Part 2



Data 260 – HW6
Eric Zhao
SID: 019141421

Message

My favorite inspection cuisine is Korean.

Send

Reply and memory used


```
{
  "reply": "Your favorite inspection cuisine is Korean.",
  "user_id": "eric",
  "session_id": "6cb5abffca8f",
  "memory_used": {
    "short_term_count": 3,
    "short_term": [
      {
        "role": "user",
        "content": "My inspection city is Gilroy."
      },
      {
        "role": "assistant",
        "content": "Your inspection city is Gilroy."
      },
      {
        "role": "user",
        "content": "My favorite inspection cuisine is Korean."
      }
    ],
    "long_term": null,
    "episodes": [
      {
        "fact": "My inspection city is Gilroy.",
        "importance": 0.9,
        "similarity": 0.5477
      }
    ]
  },
  "usage": {
    "prompt_tokens": 1130,
    "completion_tokens": 8,
    "primer_tokens": 1073,
    "long_term_tokens": 0,
    "short_term_tokens": 33,
    "episodic_tokens": 13,
    "latency_ms": 1990
```

Message

My inspection weekday is Thursday.

Send

Reply and memory used


```
{
  "reply": "Your inspection weekday is Thursday.",
  "user_id": "eric",
  "session_id": "6cb5abffca8f",
  "memory_used": {
    "short_term_count": 5,
    "short_term": [
      {
        "role": "user",
        "content": "My inspection city is Gilroy."
      },
      {
        "role": "assistant",
        "content": "Your inspection city is Gilroy."
      },
      {
        "role": "user",
        "content": "My favorite inspection cuisine is Korean."
      },
      {
        "role": "assistant",
        "content": "Your favorite inspection cuisine is Korean."
      },
      {
        "role": "user",
        "content": "My inspection weekday is Thursday."
      }
    ],
    "long_term": null,
    "episodes": [
      {
        "fact": "My inspection city is Gilroy.",
        "importance": 0.9,
        "similarity": 0.6
      },
      {
        "fact": "My favorite inspection cuisine is Korean.",
```

```
        "fact": "My favorite inspection cuisine is Korean.",
        "importance": 0.9,
        "similarity": 0.5477
      },
      {
        "fact": "The user's favorite inspection cuisine is Korean.",
        "importance": 0.8,
        "similarity": 0.3162
      }
    ]
  },
  "usage": {
    "prompt_tokens": 1187,
    "completion_tokens": 7,
    "primer_tokens": 1072,
    "long_term_tokens": 0,
    "short_term_tokens": 59,
    "episodic_tokens": 47,
    "latency_ms": 2029
  },
  "consolidation_due": true
}
```

Message

My critical deduction is 7 points.

Send

Reply and memory used


```
{
  "reply": "Your critical deduction is 7 points.",
  "user_id": "eric",
  "session_id": "6cb5abffca8f",
  "memory_used": {
    "short_term_count": 7,
    "short_term": [
      {
        "role": "user",
        "content": "My inspection city is Gilroy."
      },
      {
        "role": "assistant",
        "content": "Your inspection city is Gilroy."
      },
      {
        "role": "user",
        "content": "My favorite inspection cuisine is Korean."
      },
      {
        "role": "assistant",
        "content": "Your favorite inspection cuisine is Korean."
      },
      {
        "role": "user",
        "content": "My inspection weekday is Thursday."
      },
      {
        "role": "assistant",
        "content": "Your inspection weekday is Thursday."
      },
      {
        "role": "user",
        "content": "My critical deduction is 7 points."
      }
    ],
```

```
    ],
    "long_term": null,
    "episodes": [
      {
        "fact": "My inspection weekday is Thursday.",
        "importance": 0.9,
        "similarity": 0.3651
      },
      {
        "fact": "My inspection city is Gilroy.",
        "importance": 0.9,
        "similarity": 0.3651
      },
      {
        "fact": "My favorite inspection cuisine is Korean.",
        "importance": 0.9,
        "similarity": 0.3333
      }
    ]
  },
  "usage": {
    "prompt_tokens": 1208,
    "completion_tokens": 9,
    "primer_tokens": 1073,
    "long_term_tokens": 0,
    "short_term_tokens": 83,
    "episodic_tokens": 43,
    "latency_ms": 2188
  },
  "consolidation_due": true
}
```

Data 260 – HW6
Eric Zhao
SID: 019141421

Message

What is my inspection city?

Send

Reply and memory used


```
{
  "reply": "Your inspection city is Gilroy.",
  "user_id": "eric",
  "session_id": "6cb5abffca8f",
  "memory_used": {
    "short_term_count": 9,
    "short_term": [
      {
        "role": "user",
        "content": "My inspection city is Gilroy."
      },
      {
        "role": "assistant",
        "content": "Your inspection city is Gilroy."
      },
      {
        "role": "user",
        "content": "My favorite inspection cuisine is Korean."
      },
      {
        "role": "assistant",
        "content": "Your favorite inspection cuisine is Korean."
      },
      {
        "role": "user",
        "content": "My inspection weekday is Thursday."
      },
      {
        "role": "assistant",
        "content": "Your inspection weekday is Thursday."
      },
      {
        "role": "user",
        "content": "My critical deduction is 7 points."
      },
      {
        "role": "assistant",
```

```
      {
        "role": "user",
        "content": "What is my inspection city?"
      }
    ],
    "long_term": null,
    "episodes": [
      {
        "fact": "My inspection city is Gilroy.",
        "importance": 0.9,
        "similarity": 0.8
      },
      {
        "fact": "My inspection weekday is Thursday.",
        "importance": 0.9,
        "similarity": 0.6
      },
      {
        "fact": "My favorite inspection cuisine is Korean.",
        "importance": 0.9,
        "similarity": 0.5477
      }
    ]
  },
  "usage": {
    "prompt_tokens": 1232,
    "completion_tokens": 8,
    "primer_tokens": 1077,
    "long_term_tokens": 0,
    "short_term_tokens": 105,
    "episodic_tokens": 43,
    "latency_ms": 3419
  },
  "consolidation_due": true
}
```

Data 260 – HW6
Eric Zhao
SID: 019141421

Message

What is my favorite inspection cuisine?

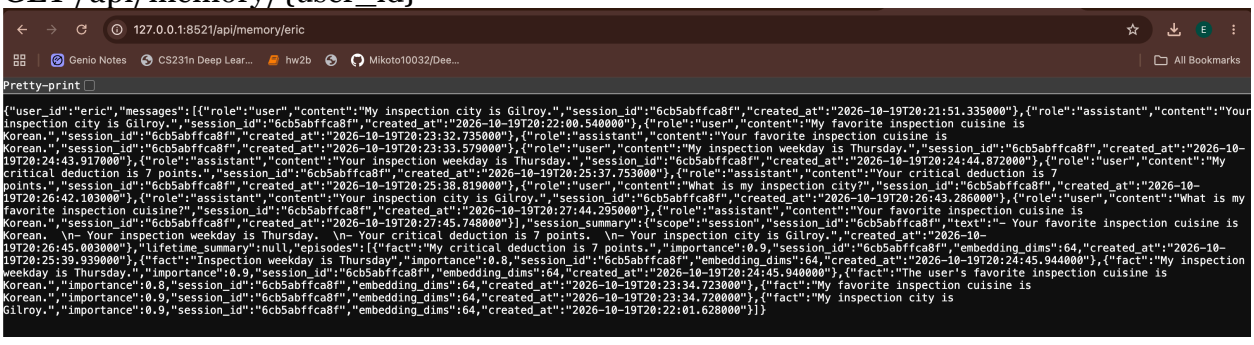
Send

Reply and memory used


```
{
  "reply": "Your favorite inspection cuisine is Korean.",
  "user_id": "eric",
  "session_id": "6cb5abffca8f",
  "memory_used": {
    "short_term_count": 11,
    "short_term": [
      {
        "role": "user",
        "content": "My inspection city is Gilroy."
      },
      {
        "role": "assistant",
        "content": "Your inspection city is Gilroy."
      },
      {
        "role": "user",
        "content": "My favorite inspection cuisine is Korean."
      },
      {
        "role": "assistant",
        "content": "Your favorite inspection cuisine is Korean."
      },
      {
        "role": "user",
        "content": "My inspection weekday is Thursday."
      },
```

```
    {
      "role": "user",
      "content": "My critical deduction is 7 points."
    },
    {
      "role": "assistant",
      "content": "Your critical deduction is 7 points."
    },
    {
      "role": "user",
      "content": "What is my inspection city?"
    },
    {
      "role": "assistant",
      "content": "Your inspection city is Gilroy."
    },
    {
      "role": "user",
      "content": "What is my favorite inspection cuisine?"
    }
  ],
  "long_term": "Session summary:\n- Your favorite inspection cuisine is Korean.  \n- Your
nspection weekday is Thursday.  \n- Your critical deduction is 7 points.  \n- Your inspectio
ity is Gilroy.",
  "episodes": [
    {
      "fact": "My favorite inspection cuisine is Korean.",
      "importance": 0.9,
      "similarity": 0.8333
    },
    {
      "fact": "The user's favorite inspection cuisine is Korean.",
      "importance": 0.8,
      "similarity": 0.5774
    },
    {
      "fact": "My inspection weekday is Thursday.",
      "importance": 0.9,
      "similarity": 0.5477
    }
  ]
},
"usage": {
  "prompt_tokens": 1296,
  "completion_tokens": 8,
  "primer_tokens": 1064,
  "long_term_tokens": 47,
  "short_term_tokens": 127,
  "episodic_tokens": 48,
  "latency_ms": 2019
```

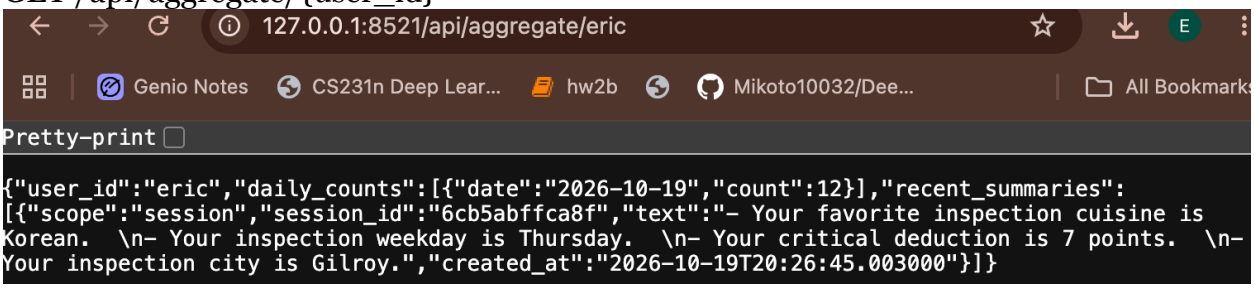

GET /api/memory/{user_id}

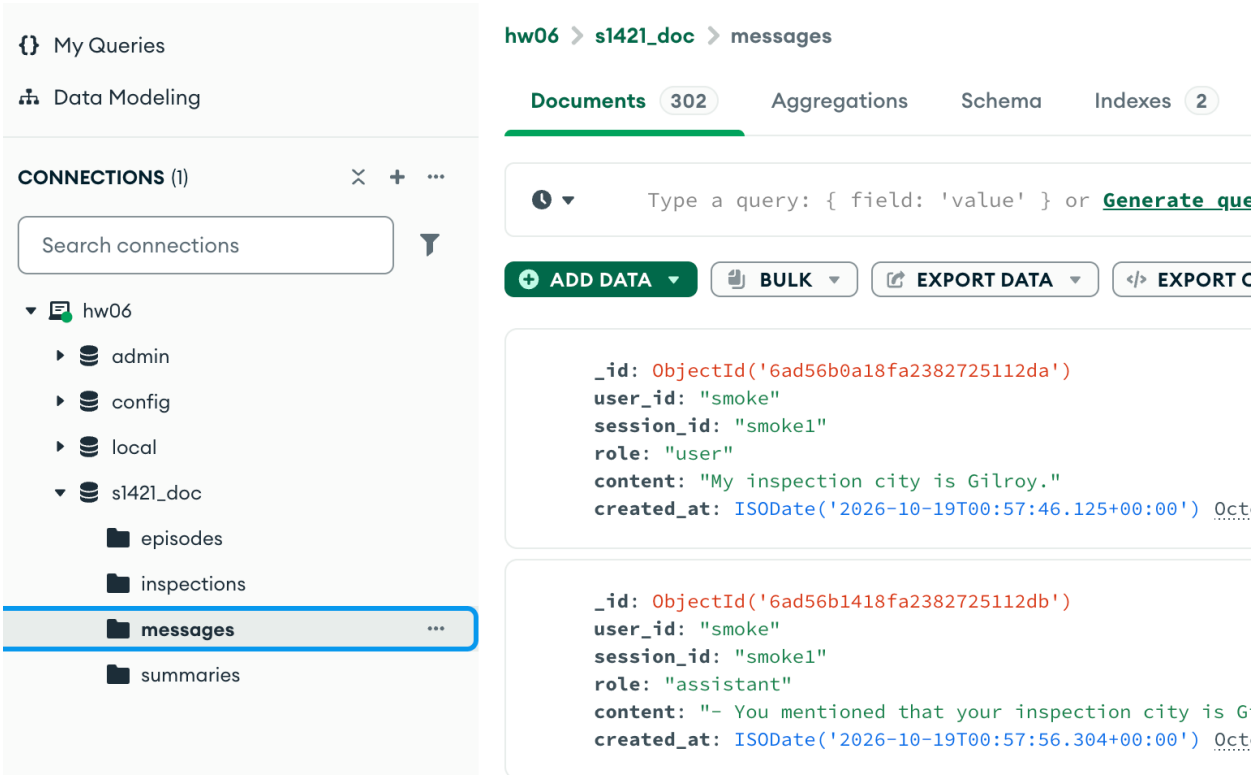



```
{"user_id":"eric","messages":[{"role":"user","content":"My inspection city is Gilroy.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:21:51.335000"},{"role":"assistant","content":"Your
inspection city is Gilroy.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:22:00.540000"},{"role":"user","content":"My favorite inspection cuisine is
Korean.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:23:32.735000"},{"role":"assistant","content":"Your favorite inspection cuisine is
Korean.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:23:33.579000"},{"role":"user","content":"My inspection weekday is Thursday.","session_id":"6cb5abffca8f","created_at":"2026-10-
19T20:24:43.917000"},{"role":"assistant","content":"Your inspection weekday is Thursday.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:24:44.872000"},{"role":"user","content":"My
critical deduction is 7 points.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:25:37.753000"},{"role":"assistant","content":"Your critical deduction is 7
points.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:25:38.819000"},{"role":"user","content":"What is my inspection city?","session_id":"6cb5abffca8f","created_at":"2026-10-
19T20:26:42.103000"},{"role":"assistant","content":"Your inspection city is Gilroy.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:26:43.286000"},{"role":"user","content":"What is my
favorite inspection cuisine?","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:27:44.295000"},{"role":"assistant","content":"Your favorite inspection cuisine is
Korean.","session_id":"6cb5abffca8f","created_at":"2026-10-19T20:27:45.748000"}],"session_summary":{"scope":"session","session_id":"6cb5abffca8f","text":"- Your favorite inspection cuisine is
Korean.  \n- Your inspection weekday is Thursday.  \n- Your critical deduction is 7 points.  \n- Your inspection city is Gilroy.","created_at":"2026-10-
19T20:26:45.003000"},"lifetime_summary":null,"episodes":[{"fact":"My critical deduction is 7 points.","importance":0.9,"session_id":"6cb5abffca8f","embedding_dims":64,"created_at":"2026-10-
19T20:25:39.939000"},{"fact":"Inspection weekday is Thursday","importance":0.8,"session_id":"6cb5abffca8f","embedding_dims":64,"created_at":"2026-10-19T20:24:45.944000"},{"fact":"My inspection
weekday is Thursday.","importance":0.9,"session_id":"6cb5abffca8f","embedding_dims":64,"created_at":"2026-10-19T20:24:45.940000"},{"fact":"The user's favorite inspection cuisine is
Korean.","importance":0.8,"session_id":"6cb5abffca8f","embedding_dims":64,"created_at":"2026-10-19T20:23:34.723000"},{"fact":"My favorite inspection cuisine is
Korean.","importance":0.9,"session_id":"6cb5abffca8f","embedding_dims":64,"created_at":"2026-10-19T20:23:34.720000"},{"fact":"My inspection city is
Gilroy.","importance":0.9,"session_id":"6cb5abffca8f","embedding_dims":64,"created_at":"2026-10-19T20:22:01.628000"}]}
```

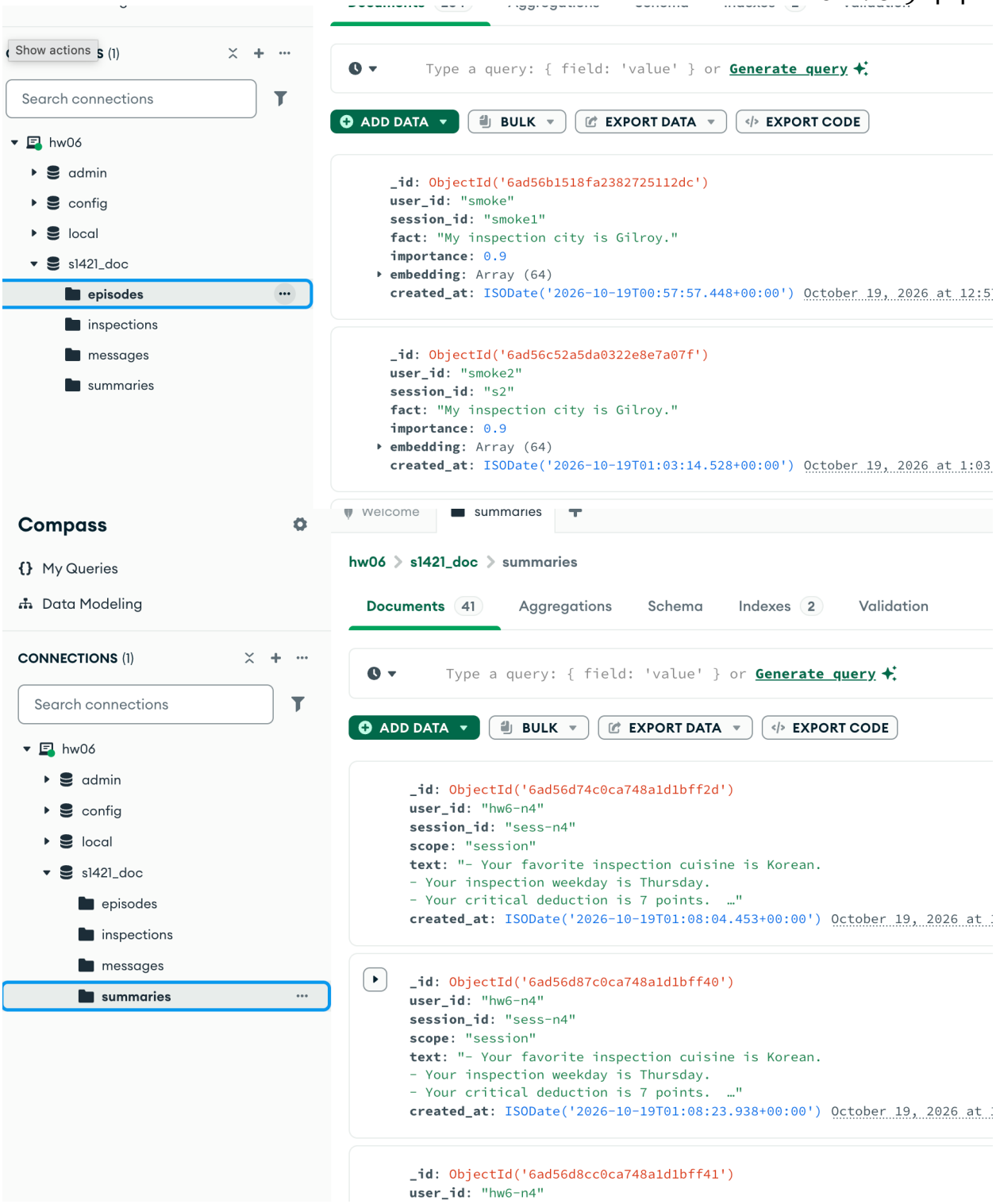
Data 260 – HW6
Eric Zhao
SID: 019141421

GET /api/aggregate/{user_id}





Data 260 – HW6
Eric Zhao
SID: 019141421




```
@router.post("/api/chat")
def chat(payload: ChatIn):
    command = payload.message.strip().split()[0].lower()
    if payload.message.strip().startswith("/"):
```

```
        if command == "/help":
            reply = help_text()
        elif command == "/memory":
            due = consolidation_due(memory_text())
            reply = load_file_memory() + f"\n\nconsolidation_due: {str(due).lower()}"
        elif command == "/compact":
            before, after = compact_memory()
            reply = f"MEMORY.md compacted: {before} content lines before, {after}
content lines after."
        elif command == "/dream":
            try:
                added = _dream(payload.user_id)
            except PyMongoError:
                raise HTTPException(status_code=503, detail="MongoDB is not
available.")
            reply = "Added to MEMORY.md: " + ("; ".join(added) if added else "nothing
new.")
        else:
            raise HTTPException(status_code=400, detail="Unknown command. Try /help.")
        return {
            "reply": reply,
            "user_id": payload.user_id,
            "session_id": payload.session_id,
            "memory_used": None,
        }
    try:
        return run_chat(
            user_id=payload.user_id,
            session_id=payload.session_id,
            message=payload.message.strip(),
            short_term_n=payload.short_term_n,
            policy=payload.policy,
            run_label=payload.run_label,
            phase=payload.phase,
        )
    except HTTPException:
        raise
    except PyMongoError:
        raise HTTPException(status_code=503, detail="MongoDB is not available.")
    except Exception as exc:
        raise HTTPException(status_code=502, detail=f"Model error:
{type(exc).__name__}: {exc}")


@router.get("/api/memory/{user_id}")
def read_memory(user_id: str):
    try:
```

```
        return memory_view(user_id)
    except PyMongoError:
        raise HTTPException(status_code=503, detail="MongoDB is not available.")


@router.get("/api/aggregate/{user_id}")
def read_aggregate(user_id: str):
    try:
        return aggregate_view(user_id)
    except PyMongoError:
        raise HTTPException(status_code=503, detail="MongoDB is not available.")
```

```
def run_chat(
    user_id: str,
    session_id: str | None,
    message: str,
    short_term_n: int | None = None,
    policy: str = "none",
    run_label: str = "live",
    phase: str = "live",
) -> dict:
    started = time.perf_counter()
    ensure_collections()
    session_id = session_id or uuid.uuid4().hex[:12]
    window_n = short_term_n or SHORT_TERM_N

    save_message(user_id, session_id, "user", message)
    loaded = load_window(user_id, session_id, window_n)
    history = loaded[:-1] if loaded and loaded[-1]["content"] == message else loaded
    history = compress_history(history, policy)
    session_summary = latest_summary(user_id, session_id, "session")
    lifetime_summary = latest_summary(user_id, None, "user")
    found = retrieve_episodes(user_id, message)

    long_parts = []
    if lifetime_summary:
        long_parts.append("Lifetime summary:\n" + lifetime_summary)
    if session_summary:
        long_parts.append("Session summary:\n" + session_summary)
    long_text = "\n\n".join(long_parts)
    episode_text = "\n".join(
        f"- {item['fact']} (importance {item['importance']})" for item in found
    )
    primer = "File memory:\n" + load_file_memory() + "\n\n" + PRIMER
    prompt = [{"role": "system", "content": primer}]
    if long_text:
        prompt.append({"role": "system", "content": long_text})
```

```
    if episode_text:
        prompt.append({"role": "system", "content": "Episodic facts:\n" +
episode_text})
    prompt.extend(history)
    prompt.append({"role": "user", "content": message})

    response = chat_model().complete(prompt)
    reply = strip_think(response.content) or "I do not know."
    save_message(user_id, session_id, "assistant", reply)
    maybe_summarize(user_id, session_id)
    store_episodes(user_id, session_id, extract_facts(message))

    used = history + [{"role": "user", "content": message}]
    sections = {
        "primer": primer,
        "long_term": long_text,
        "short_term": _transcript(used),
        "episodic": episode_text,
        "current": message,
    }
    shares = _section_tokens(sections, response.input_tokens)
    latency_ms = int((time.perf_counter() - started) * 1000)
    _log_turn({
        "ts": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
        "run_label": run_label,
        "phase": phase,
        "user_id": user_id,
        "session_id": session_id,
        "prompt_tokens": response.input_tokens,
        "completion_tokens": response.output_tokens,
        "primer_tokens": shares["primer"],
        "long_term_tokens": shares["long_term"],
        "short_term_tokens": shares["short_term"],
        "episodic_tokens": shares["episodic"],
        "latency_ms": latency_ms,
        "policy": policy,
        "short_term_n": window_n,
    })
    return {
        "reply": reply,
        "user_id": user_id,
        "session_id": session_id,
        "memory_used": {
            "short_term_count": len(used),
            "short_term": used,
            "long_term": long_text or None,
            "episodes": found,
```

```
        },
        "usage": {
            "prompt_tokens": response.input_tokens,
            "completion_tokens": response.output_tokens,
            "primer_tokens": shares["primer"],
            "long_term_tokens": shares["long_term"],
            "short_term_tokens": shares["short_term"],
            "episodic_tokens": shares["episodic"],
            "latency_ms": latency_ms,
        },
        "consolidation_due": consolidation_due(memory_text()),
    }
```

```
def extract_facts(message: str) -> list[dict]:
    facts = []
    if message.startswith("My "):
        facts.append({"fact": message.strip()[:200], "importance": 0.9})
    try:
        reply = _complete(json_model(), [
            {"role": "system", "content": "Extract durable facts. Return JSON only."},
            {"role": "user", "content": (
                "Message:\n" + message + "\n\n"
                'Return {"facts":[{"fact":"short text","importance":0.8}]} with at
most 3 facts. '
                "importance is a number from 0 to 1. Use an empty list when nothing
should be stored."
            )},
        ])
        payload = parse_json_object(reply)
        for item in payload.get("facts") or []:
            if len(facts) >= 3:
                break
            fact = str(item.get("fact", "")).strip()[:200]
            if not fact or any(existing["fact"] == fact for existing in facts):
                continue
            try:
                importance = float(item.get("importance", 0.5))
            except (TypeError, ValueError):
                importance = 0.5
            importance = min(1.0, max(0.0, importance))
            facts.append({"fact": fact, "importance": importance})
    except Exception:
        if not facts and len(message.split()) >= 6:
            facts.append({"fact": message.strip()[:200], "importance": 0.5})
    return facts[:3]
```

```
def maybe_summarize(user_id: str, session_id: str) -> None:
    user_count = messages().count_documents({
        "user_id": user_id, "session_id": session_id, "role": "user",
    })
    if user_count == 0 or user_count % SUMMARIZE_EVERY_USER_MSGS != 0:
        return
    recent = load_window(user_id, session_id, 8)
    previous = latest_summary(user_id, session_id, "session") or "(none)"
    try:
        text = _complete(chat_model(), [
            {"role": "system", "content": "Summarize restaurant-inspection
conversations."},
            {"role": "user", "content": (
                "Update the session summary in at most 8 bullet lines. "
                "Keep every name and number from the previous summary.\n\n"
                f"Previous summary:\n{previous}\n\nRecent
messages:\n{_transcript(recent)}"
            )},
        ])
    except Exception:
        text = "\n".join(f"- {row['content'][:120]}" for row in recent if row["role"]
== "user")
    if not text:
        return
    summaries().insert_one({
        "user_id": user_id,
        "session_id": session_id,
        "scope": "session",
        "text": text[:2000],
        "created_at": now_utc(),
    })
    session_count = summaries().count_documents({"user_id": user_id, "scope":
"session"})
    if session_count % LIFETIME_EVERY == 0:
        write_lifetime(user_id)
```

```
# Five probes asked after the same 30-message conversation.
# A probe scores 5 when every expect string is present, 3 when some are, and 1 when
none are.
# match "word" requires a whole word, so "8" does not match "80".
probes:
  - id: city
    question: "What is my inspection city?"
    expect: ["gilroy"]
    match: substring
    rubric: "5 if the answer says Gilroy, 1 if it does not."
  - id: cuisine
```

```
    question: "What is my favorite inspection cuisine?"
    expect: ["korean"]
    match: substring
    rubric: "5 if the answer says Korean, 1 if it does not."
  - id: weekday
    question: "What is my inspection weekday?"
    expect: ["thursday"]
    match: substring
    rubric: "5 if the answer says Thursday, 1 if it does not."
  - id: deduction
    question: "What is my critical deduction?"
    expect: ["7"]
    match: word
    rubric: "5 if the answer gives 7 as a whole word, 1 if it does not."
  - id: noodle
    question: "What is the Lantern Noodle 150 score and the next step?"
    expect: ["91", "return"]
    match: substring
    rubric: "5 if the answer has both 91 and a return visit, 3 if it has only one, 1
if it has neither."
```

Part 3

```
echo "Eric Zhao (SID4 1421)"
cat reports/hw06/raw/memory_cost_summary.csv
echo
tail -n 5 reports/hw06/RUN_LOG.txt
Eric Zhao (SID4 1421)
setting,short_term_n,policy,mean_prompt_tokens,mean_completion_tokens,mean_latency_ms,mean_probe_score,cost_10000_conversations_usd
n4,4,none,1411.6,32.2,5632.1,5.0,471.78
n12,12,none,1618.0,32.3,6145.9,5.0,533.85
n30,30,none,1936.8,32.0,7416.9,5.0,629.04
compress,30,compress,1648.8,36.2,6280.8,5.0,548.94

[2026-10-18 18:21:59] part 3 short-term sweep
n4: prompt=1411.6 completion=32.2 latency_ms=5632.1 probe=5.0 cost_usd=471.78
n12: prompt=1618.0 completion=32.3 latency_ms=6145.9 probe=5.0 cost_usd=533.85
n30: prompt=1936.8 completion=32.0 latency_ms=7416.9 probe=5.0 cost_usd=629.04
compress: prompt=1648.8 completion=36.2 latency_ms=6280.8 probe=5.0 cost_usd=548.94
(base) eric@Eric-Mac data260 %
```

All four settings scored 5.0 on the five fixed probes, so I ranked them by mean prompt tokens per turn (and the matching latency and dollar cost). The best setting is SHORT_TERM_N=4: it used the fewest prompt tokens (1411.6), the lowest mean latency (5632 ms), and the lowest estimated cost for 10,000 thirty-turn conversations ($471.78), while still recovering the early personal facts from episodic memory and summaries. The worst setting is SHORT_TERM_N=30 with no compression: it kept the same probe score but used the most tokens (1936.8), the highest latency (7417 ms), and the highest cost ($629.04). The compression policy sat in between: relative to N=30 it cut about 288 prompt tokens per turn (14.9%) with no drop in probe quality, but it was still more expensive than N=4, so it is neither the best nor the worst.

Data 260 – HW6
Eric Zhao
SID: 019141421

Part 4

Before:

```
<!-- consolidated: 2026-01-01T00:00:00Z -->
# Durable facts
- repo_owner: Eric Zhao
- repo_owner: Eric Zhao
- repo_owner: Eric Zhao
- repo_owner: Eric Zhao
- repo_owner: Eric Zhao
- repo_owner: Eric Zhao
- repo_owner: Eric Zhao
- repo_owner: Eric Zhao
- hand_sink: soap
- hand_sink: soap
- hand_sink: soap
- hand_sink: soap
- hand_sink: soap
- hand_sink: soap
- hand_sink: soap
- hand_sink: soap
- report_dir: reports/hw06
- report_dir: reports/hw06
- report_dir: reports/hw06
- report_dir: reports/hw06
- report_dir: reports/hw06
- report_dir: reports/hw06
- report_dir: reports/hw06
- report_dir: reports/hw06
- default_score: 80
- default_score: 100
- glove_rule: reuse
- glove_rule: replace
- port_base: 8521
- port_base: 8000
- note_01: Each inspection has a code shaped like INS-00001.
- note_02: A score starts at 100 before deductions.
- note_03: Minor violations are worth less than major ones.
- note_04: Deleting an owner who still has inspections is refused.
- note_05: Owner email addresses must be unique.
- note_06: The document database is separate from the MySQL database.
- note_07: Cuisine is stored on the inspection, not on the owner.
- note_08: Violations are removed when their inspection is removed.
- note_09: Session cookies expire after 30 minutes.
- note_10: The chat route does not require a login cookie.
- note_11: Short-term memory is a sliding window of recent messages.
- note_12: Lifetime summaries have no session id.
```

```
- note_13: Episodic facts keep an importance score between 0 and 1.
- note_14: Probe questions are fixed before the memory test runs.
- note_15: The local model is qwen3:8b.
- note_16: Reports for this homework live under reports/hw06.
- note_17: A second visit means the restaurant is not finished yet.
- note_18: Closed status means the inspection is finished.
- note_19: Open status is the default for a new inspection document.
- note_20: Notes on an inspection document are optional.
- note_21: The restaurant name on a document cannot exceed 100 characters.
- note_22: Daily message counts are grouped by the date of the message.
- note_23: File memory is loaded from AGENT.md and MEMORY.md.
```

```
(base) eric@Eric-Mac data260 % cd /Users/eric/data260
echo "Eric Zhao (SID4 1421)"
curl -s http://127.0.0.1:8521/api/chat \
  -H 'Content-Type: application/json' \
  -d '{"user_id":"eric","message":"/compact"}'
echo
wc -l MEMORY.md
Eric Zhao (SID4 1421)
{"reply":"MEMORY.md compacted: 54 content lines before, 33 content lines after.","user_id":"eric","session_id":null,"memory_used":null}
      34 MEMORY.md
(base) eric@Eric-Mac data260 %
```

After:

```
<!-- consolidated: 2026-10-19T20:40:23Z -->
# Durable facts
- repo_owner: Eric Zhao
- hand_sink: soap
- report_dir: reports/hw06
- default_score: 100
- glove_rule: replace
- port_base: 8000
- note_01: Each inspection has a code shaped like INS-00001.
- note_02: A score starts at 100 before deductions.
- note_03: Minor violations are worth less than major ones.
- note_04: Deleting an owner who still has inspections is refused.
- note_05: Owner email addresses must be unique.
- note_06: The document database is separate from the MySQL database.
- note_07: Cuisine is stored on the inspection, not on the owner.
- note_08: Violations are removed when their inspection is removed.
- note_09: Session cookies expire after 30 minutes.
- note_10: The chat route does not require a login cookie.
- note_11: Short-term memory is a sliding window of recent messages.
- note_12: Lifetime summaries have no session id.
- note_13: Episodic facts keep an importance score between 0 and 1.
- note_14: Probe questions are fixed before the memory test runs.
- note_15: The local model is qwen3:8b.
- note_16: Reports for this homework live under reports/hw06.
- note_17: A second visit means the restaurant is not finished yet.
- note_18: Closed status means the inspection is finished.
```

```
- note_19: Open status is the default for a new inspection document.
- note_20: Notes on an inspection document are optional.
- note_21: The restaurant name on a document cannot exceed 100 characters.
- note_22: Daily message counts are grouped by the date of the message.
- note_23: File memory is loaded from AGENT.md and MEMORY.md.
- resolved default_score: kept "100" and dropped "80".
- resolved glove_rule: kept "replace" and dropped "reuse".
- resolved port_base: kept "8000" and dropped "8521".
```

```
# Project instructions

These notes stay with the project. The bullet-only rule above applies only when the
user asks for a code review.

- Project: this repository is a local restaurant-inspection system for San Jose. An
owner has many inspections, and an inspection has many violations.
- Stack: FastAPI on port 8521, SQLAlchemy with MySQL database s1421_rel, PyMongo with
MongoDB database s1421_doc, and Ollama model qwen3:8b through src/model_client.py.
- Convention: a Python helper uses a snake_case name, a one-line docstring, and type
hints on every argument and the return value.
- Testing: smoke tests live in code/verify_hw0N.py and check behavior, not the exact
wording of a model reply.
- Prohibited: a helper must not print or return an owner's phone number or email
address, and the MongoDB document routes must not delete MySQL rows.
- Scores: a function that classifies an inspection score raises ValueError when the
score is below 0 or above 100.
```

```python
def inspection_score_label(score: int) -> str:
    """Return a short label for an inspection score from 0 to 100."""
    if score < 0 or score > 100:
        raise ValueError("score must be between 0 and 100")
    if score >= 90:
        return "good"
    if score >= 70:
        return "fair"
    return "poor"
```

```python
def consolidation_due(text: str, now: datetime | None = None) -> bool:
    if content_line_count(text) > 40:
        return True
    stamped = last_consolidated_at(text)
    if stamped is None:
        return True
    current = now or datetime.now(timezone.utc)
    return current - stamped > timedelta(hours=24)
```

```
def consolidate_memory(text: str, now: datetime | None = None) -> tuple[str, int,
int]:
    """Merge duplicate facts and write an explicit line for every conflict."""
    before = content_line_count(text)
    grouped: dict[str, list[str]] = {}
    order: list[str] = []
    extras: list[str] = []
    for line in content_lines(text):
        if line.startswith("#") or line.startswith("- resolved "):
            continue
        match = KEY_RE.match(line)
        if not match:
            extras.append(line)
            continue
        key, value = match.group(1), match.group(2).strip()
        if key not in grouped:
            order.append(key)
            grouped[key] = []
        if value not in grouped[key]:
            grouped[key].append(value)

    current = now or datetime.now(timezone.utc)
    out = [f"<!-- consolidated: {_stamp(current)} -->", "# Durable facts"]
    resolutions = []
    for key in order:
        values = grouped[key]
        kept = values[-1]
        out.append(f"- {key}: {kept}")
        if len(values) > 1:
            dropped = ", ".join(f'"{item}"' for item in values[:-1])
            resolutions.append(f'- resolved {key}: kept "{kept}" and dropped
{dropped}.')
    out.extend(resolutions)
    out.extend(extras)
    rendered = "\n".join(out) + "\n"
    return rendered, before, content_line_count(rendered)
```

```
@router.post("/api/chat")
def chat(payload: ChatIn):
    command = payload.message.strip().split()[0].lower()
    if payload.message.strip().startswith("/"):
        if command == "/help":
            reply = help_text()
        elif command == "/memory":
            due = consolidation_due(memory_text())
            reply = load_file_memory() + f"\n\nconsolidation_due: {str(due).lower()}"
        elif command == "/compact":
```

```
            before, after = compact_memory()
            reply = f"MEMORY.md compacted: {before} content lines before, {after}
content lines after."
        elif command == "/dream":
            try:
                added = _dream(payload.user_id)
            except PyMongoError:
                raise HTTPException(status_code=503, detail="MongoDB is not
available.")
            reply = "Added to MEMORY.md: " + ("; ".join(added) if added else "nothing
new.")
        else:
            raise HTTPException(status_code=400, detail="Unknown command. Try /help.")
        return {
            "reply": reply,
            "user_id": payload.user_id,
            "session_id": payload.session_id,
            "memory_used": None,
        }
    try:
        return run_chat(
            user_id=payload.user_id,
            session_id=payload.session_id,
            message=payload.message.strip(),
            short_term_n=payload.short_term_n,
            policy=payload.policy,
            run_label=payload.run_label,
            phase=payload.phase,
        )
    except HTTPException:
        raise
    except PyMongoError:
        raise HTTPException(status_code=503, detail="MongoDB is not available.")
    except Exception as exc:
        raise HTTPException(status_code=502, detail=f"Model error:
{type(exc).__name__}: {exc}")
```